从一条 SQL 认识 seekdb

语义 · 计划 · 执行 · 实验

数据库技术课程

2026 年 09 月 30 日

seekdb：在关系型数据库内核上加入 AI 能力

关系型数据库主干
用表和 Schema 管理数据；用 SQL 表达查询；由 Parser、Resolver、Optimizer 和 Executor 完成语义理解、计划选择与执行。

+

面向 AI 应用的扩展
在 SQL 中增加 AI Function，并让 JSON、文本、向量等数据继续使用同一套类型、表达式、计划和执行机制。

没有另起一套数据库
AI 能力不是绕开关系型引擎的旁路服务；它要服从 SQL 的类型、NULL、错误和执行语义。

本课从共同主干入手
先追踪一条普通 SQL 怎样被理解和执行，再用同一套方法定位 Lab 中的 AI Function。

这节课的范围：只追踪三段主干



建立直觉
看懂每一阶段的输入、输出以及职责边界。

学会定位
根据“最早在哪里变错”缩小到正确源码目录。

完成闭环
克隆、编译、运行、修改、测试和记录。

本课暂不展开
网络协议与会话、DAS、LS / Tablet、MemTable / SSTable、MVCC 与日志。它们很重要，但不是后续两个 Lab 的第一落点。

第一部分 · 先建立数据库系统的基础直觉

数据库不是“替你循环一遍文件”

声明意图
用户用 SQL 描述“要什么”，不描述每一步怎样循环。

→

理解并选择
数据库检查语义，比较执行办法，再生成算子树。

→

按约定执行
处理类型、NULL、顺序、错误与资源，产出稳定结果。

为什么要分层
每一层把一种表示变成下一种表示，并保证自己的不变量。

为什么利于定位
当输出错误时，寻找“第一次失真”的转换层，比追最后一条日志更有效。

我们的观察 把数据库先理解成一台“受约束的翻译器 + 执行器”。

全课只用这一张表和这一条查询

```sql
CREATE TABLE students (
  id BIGINT PRIMARY KEY,
  name VARCHAR(64),
  score INT
);

SELECT id, name
FROM students
WHERE score >= 80 AND 1 = 1
ORDER BY score DESC
LIMIT 3;
```

id	name	score
1	Alice	91
2	Bob	76
3	Chen	88
4	Dora	95

预期结果
`(4, Dora) · (1, Alice) · (3, Chen)`

投影 id, name　过滤 score ≥ 80　排序 score DESC　限制 3 行

`AND 1 = 1` 是故意保留的冗余条件，后面用它观察 Rewrite。

一条 SQL 的路线图：只看语义、计划、执行



语义
Parser + Resolver：这句话结构正确吗？名字与类型是什么意思？

计划
Rewrite + Optimizer + CodeGen：怎样把意图组织成可执行步骤？

执行
Executor + Expression：逐行或成批地产生结果。

观察方法
每经过一站，都问两件事：输入长什么样？输出比输入多了什么信息？

第二部分 · 一条 SQL 的语义、计划与执行

先看全貌：同一句话不断换“表示形式”

1	**SQL string**	用户可读的字符序列。
2	**Parse tree**	语法节点：SELECT、表、列、谓词、排序、LIMIT。
3	**Resolved statement**	表列绑定到 ID，表达式拥有类型。
4	**Logical plan**	用关系算子描述“做什么”。
5	**Physical operators**	具体算子和表达式可被 open / next / close。
6	**Row / Batch**	执行中间数据；最终投影为结果行。

第 1 站：Parser 把字符变成语法树

输入 · SQL string

```
SELECT id, name
FROM students
WHERE score >= 80 AND 1 = 1
ORDER BY score DESC
LIMIT 3;
```

→

输出 · Parse tree

```
SELECT
├─ projection: id, name
├─ table: students
├─ predicate: score >= 80 AND 1 = 1
├─ order: score DESC
└─ limit: 3
```

这一站增加了什么
关键字、括号和运算符不再是一串字符，而是有父子关系的语法节点。

这一站还不知道什么
students 是否存在、score 是什么类型、比较是否合法。

实现目录：`src/sql/parser/`　　主流程入口：`src/sql/ob_sql.cpp`

第 2 站：Resolver 让名字和表达式拥有含义

输入 · Parse tree

```
SELECT
├─ projection: id, name
├─ table: students
├─ predicate: score >= 80 AND 1 = 1
├─ order: score DESC
└─ limit: 3
```

→

输出 · Resolved statement

```
SELECT stmt
├─ id, name → typed column_id
├─ students → table_id
├─ predicate → BOOLEAN typed expr
├─ order key → score(INT)
└─ limit → INT datum(3)
```

名称绑定
students、id、name、score 分别指向谁？

类型推导
score 和 80 能否比较？结果类型是什么？

合法性检查
列是否存在或歧义？表达式是否满足 SQL 规则？

实现目录：`src/sql/resolver/`　Schema 依赖：`src/share/schema/`

第 3 站：Rewrite 做保持语义的等价变换

输入 · 冗余谓词

```
score >= 80 AND 1 = 1
```

右侧恒为 TRUE。

→

输出 · 化简后的谓词

```
score >= 80
```

结果集合不变，后续结构更简单。

可以改变

表达式形状、谓词位置、子查询形式，以及后续优化更容易处理的结构。

不能改变

结果、NULL 语义、重复行、顺序约束与错误边界。

实现目录：`src/sql/rewrite/`

第 4 站：Optimizer 把逻辑关系组织成计划

输入 · Logical requirements

```
filter score >= 80
order by score DESC
limit 3
project id, name
```

→

输出 · Chosen logical plan

```
PROJECT(id,name)
└─ LIMIT(3)
   └─ SORT(score DESC)
      └─ TABLE ACCESS
         filter: score >= 80
```

它比较什么
访问路径、算子顺序和实现候选，并估算行数、CPU、IO 与内存成本。

它不做什么
Optimizer 不亲自计算 Dora、Alice、Chen；它选择之后由哪些算子计算。

实现目录：`src/sql/optimizer/`　　主流程入口：`src/sql/ob_sql.cpp`

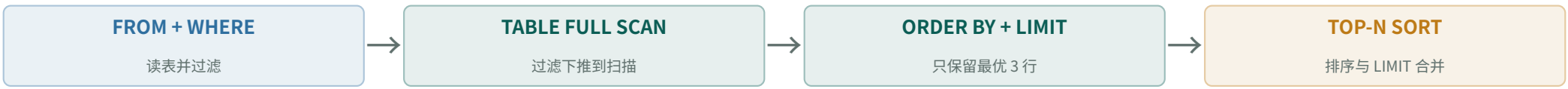
用 EXPLAIN 看计划，不靠脑补

```
EXPLAIN
SELECT id, name
FROM students
WHERE score >= 80 AND 1 = 1
ORDER BY score DESC
LIMIT 3;

=====================================================
|ID|OPERATOR         |NAME    |EST.ROWS|EST.TIME(us)|
-----------------------------------------------------
|0 |TOP-N SORT       |        |3       |3           |
|1 |└─TABLE FULL SCAN|students|4       |3           |
=====================================================
Outputs & filters:
  0 - output([students.id], [students.name]), filter(nil), rowset=16
      sort_keys([students.score, DESC]), topn(3)
  1 - output([students.id], [students.score], [students.name]),
      filter([students.score >= 80]), rowset=16
      access([students.id], [students.score], [students.name]), partitions(p0)
      range_key([students.id]), range(MIN ; MAX)always true
```

实测产物：seekdb 1.4.0.0 · revision `1-834bbee1e…`；`1 = 1` 已不再出现在执行计划中。

为什么计划里只有 SORT 和 SCAN?



为什么没有 FILTER

`score >= 80` 被下推到 TABLE FULL SCAN，扫描每一行时就完成过滤，不必再建立独立 FILTER 算子。

为什么没有 LIMIT

`ORDER BY score DESC LIMIT 3` 被实现为 TOP-N SORT：维护分数最高的 3 行，不需要先完整排序再 LIMIT。

为什么没有 PROJECT

`SELECT id, name` 记录在根算子的 `output` 表达式中；简单投影不一定需要独立 PROJECT 算子。

为什么是 FULL SCAN

本例没有可用于 `score >= 80` 的索引；主键范围是 MIN 到 MAX，所以需要检查整张表。

我们的观察 SQL 的每个子句不一定对应一个独立算子；优化器可以消除、下推或合并工作。`1 = 1` 也已在 Rewrite 阶段消失。

怎样读 Outputs & filters?

字段	它告诉我们什么
`0 - output(id,name)`	TOP-N SORT 最终向客户端交出 id、name；score 只用于排序，不在最终结果中。
`filter(nil)`	SORT 本身没有额外过滤条件。
`sort_keys(score,DESC), topn(3)`	按 score 降序，并只保留前 3 行。
`1 - output(id,score,name)`	SCAN 要把 score 交给上层排序，所以它比最终结果多输出一列。
`filter(score >= 80)`	谓词在扫描阶段执行，这就是过滤下推。
`access(id,score,name), partitions(p0)`	从 p0 分区实际读取这三列。
`range(MIN ; MAX) always true`	主键范围没有缩小，是完整范围，因此显示 TABLE FULL SCAN。

EST.ROWS
优化器估算该算子会输出多少行，不是实际执行后统计的行数；TOP-N 根节点受 LIMIT 约束，因此估算为 3。

EST.TIME(us)
优化器用于比较候选计划的估算时间，不是本次查询真实测得的墙钟耗时。

rowset=16
这两个物理算子支持向量化执行，优化器为它们记录的批处理行数是 16；它不是估算行数，估算行数在上方 `EST.ROWS` 列中。

第 5 站：Code Generation 生成可运行的物理算子

输入 · Logical operators

```
LogicalSort
LogicalTableScan
RawExpr(score >= 80)
```

→

输出 · Operator spec / expr

```
SortSpec
TableScanSpec
Expr evaluation metadata
```

成为本次执行可实例化的算子描述。

为什么还要转换

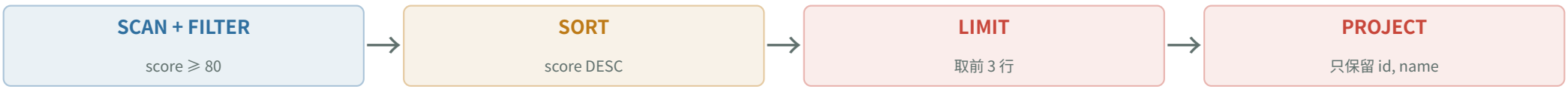
逻辑计划强调关系；物理算子还要确定运行接口、表达式元数据与执行所需状态。

定位提示

EXPLAIN 合理但真正执行的算子或表达式挂接不对，要检查生成边界。

实现目录：`src/sql/code_generator/`　物理算子定义：`src/sql/engine/`

第 6 站：Executor 让数据沿算子树流动



① 输入行
`Alice 91 · Bob 76 · Chen 88 · Dora 95`

② 过滤后
`Alice 91 · Chen 88 · Dora 95`

③ 排序后
`Dora 95 · Alice 91 · Chen 88`

④ 投影 / LIMIT 后
`(4,Dora) · (1,Alice) · (3,Chen)`

实现目录：`src/sql/executor/`　　算子实现：`src/sql/engine/`

调试主线：寻找“第一次形式不对”的位置

你观察到的现象	先看哪一站	先验证什么
新语法无法识别	**Parser**	Parse tree 是否形成
列名或参数类型错误	**Resolver**	绑定对象与表达式类型
等价改写后结果变了	**Rewrite**	改写前后语义是否一致
计划选择不符合预期	**Optimizer / CodeGen**	EXPLAIN 与物理算子
Row 正确、Batch 错位	**Executor / Expr**	skip、evaluated、row index

核心问题
上一种表示还是对的，下一种表示第一次错了——负责这次转换的组件就是首要排查点。

第三部分 · 从源码构建到服务器查询

本课采用一条可验证的源码路径



为什么采用这条路径

`build_release/src/observer/seekdb` 就是刚由本地源码生成的产物；从启动、导入到查询，每一步都能直接验证本次 Lab 修改。

第一步：克隆并编译本地源码




```
git clone https://github.com/oceanbase/seekdb.git
cd seekdb

./build.sh release --init --make

./build_release/src/observer/seekdb -V
```

第二步：按照官方流程部署本地实例

```
./tools/deploy/obd.sh prepare -p /tmp/obtest

./tools/deploy/obd.sh deploy \
  -c ./tools/deploy/single.yaml

# 查看这份配置实际使用的 SQL 端口
rg -n "mysql_port:" tools/deploy/single.yaml
```

prepare
在 `/tmp/obtest` 准备隔离的本地部署环境，并使用刚编译出的 seekdb。

deploy
读取 `single.yaml`，初始化并启动名为 `single` 的单机实例。

连接端口
当前仓库的 `single.yaml` 配置为 `mysql_port: 10100`；不要照抄文档中的示例端口，配置变化后应以文件中的值为准。

第三步：把示例数据写成 students.sql

```sql
CREATE DATABASE IF NOT EXISTS seekdb_course;
USE seekdb_course;
DROP TABLE IF EXISTS students;
CREATE TABLE students (
  id BIGINT PRIMARY KEY,
  name VARCHAR(64),
  score INT
);
INSERT INTO students VALUES
  (1, 'Alice', 91),
  (2, 'Bob',   76),
  (3, 'Chen',  88),
  (4, 'Dora',  95);
```

文件职责

可重复创建表并写入 4 行固定数据，让每位同学得到同一个输入。

文件位置

把 `students.sql` 放在仓库根目录或个人练习目录；`/tmp/obtest` 留给 OBD 管理本地部署文件。

第四步：在终端 B 连接并导入 SQL 文件

```
# 先确认服务可连接
./deps/3rd/u01/obclient/bin/obclient \
  -h127.0.0.1 -P10100 -uroot -Doceanbase -A \
  -e "SELECT VERSION();"

# 再把整个 SQL 文件交给客户端执行
./deps/3rd/u01/obclient/bin/obclient \
  -h127.0.0.1 -P10100 -uroot -A \
  < students.sql
```

1. **connect**
   确认地址、端口与用户。
2. **version**
   确认连接的是 seekdb。
3. **import**
   由 obclient 顺序执行文件。

第五步：执行查询并核对结果

```
$ ./deps/3rd/u01/obclient/bin/obclient \
    -h127.0.0.1 -P10100 -uroot -A

obclient> USE seekdb_course;
obclient> SELECT id, name
    -> FROM students
    -> WHERE score >= 80 AND 1 = 1
    -> ORDER BY score DESC
    -> LIMIT 3;
```

实际结果

```
id  name
4   Dora
1   Alice
3   Chen
```

端口

这里使用当前 `single.yaml` 中的 `10100`；若配置不同，两处连接命令都要同步替换。

结果不对时

先分别查询全表、过滤结果和排序结果，找到最早偏离预期的一步。

停止并清理

```
./tools/deploy/obd.sh destroy --rm -n single
```

只对本教程创建的临时实例执行。

我们的观察 现在已经走通：当前源码 → 当前二进制 → 当前服务进程 → SQL 文件 → 查询结果。

第四部分 · 怎样在 Lab 中找到并修改正确位置

一页源码地图：只保留本课会用到的主干

模块	负责哪次“形式转换”	目录
Parser	SQL string → Parse tree	`src/sql/parser/`
Resolver	Parse tree → Resolved stmt	`src/sql/resolver/`
Rewrite	Resolved stmt → 等价新形式	`src/sql/rewrite/`
Optimizer	语义语句 → Logical plan	`src/sql/optimizer/`
CodeGen	Logical plan → Physical spec	`src/sql/code_generator/`
Executor	Physical operators → Row / Batch	`src/sql/executor/ · engine/`
Expression	Input datum → Output datum	`src/sql/engine/expr/`

先从职责选目录，再从具体符号建立调用链；不要从头通读几百万行源码。

从需求描述反推第一批检查位置

需求或失败	优先层次	第一批证据
增加或调整 SQL 写法	**Parser**	grammar、ParseNode、parser test
函数参数 / 返回类型异常	**Resolver / Expr**	类型推导、NULL、datum
计划中缺少新访问方式	**Optimizer / CodeGen**	EXPLAIN、logical op、spec
Row 正常但 Batch 错位	**Engine / Expr**	skip、evaluated、row index
文件列无法参与 SQL	**Resolver → FileScan**	动态 Schema、scan output、rescan

定位原则
先证明确切是哪一次表示转换出错，再进入负责该转换的代码。

用 rg 建立最短调用链

rg 是什么

rg 是 ripgrep：从指定目录递归搜索文本，默认遵守 .gitignore，并把匹配文件、行号和内容打印出来。它适合在陌生仓库中由“函数名或类名”反查定义、调用者和测试。

基本格式

`rg [选项] "搜索模式" [目录]`

搜索模式可以是普通字符串，也可以是正则表达式；目录省略时从当前目录开始。

```
# 1. 先按文件名缩小范围
rg --files src/ | rg "ai_prompt"

# 2. 找定义和全部引用；| 表示“或”
rg -n "ObExprAIPrompt|N_AI_PROMPT" src/sql/

# 3. 连同命中位置前后 3 行一起看
rg -n -C 3 "calc_result_typeN" \
  src/sql/engine/expr/

# 4. 到测试目录寻找已有用例
rg -n "ai_prompt\\(" tools/deploy/mysql_test/
```

常用选项

- `-n` 显示行号
- `-C 3` 显示上下文
- `-l` 只列文件名
- `-g '*.cpp'` 限定文件
- `--files` 只列仓库文件

我们的观察 推荐顺序：先找实现文件，再找核心符号的定义与引用，最后找最近的测试；每次搜索都让范围更小。